



Code Variation as a Defense

University of Texas at Dallas
Erick Bauman
Huibo Wang
Travis Wright



Overview

1. Background
2. The Problem
3. Approach
4. Progress
5. Future Plans
6. Challenges



Background

- Buffer overflows are a popular attack vector
- Initial attacks could inject code directly (e.g. write shellcode to the stack)
- Later, W $\oplus$ X ensured that code could not be simultaneously writable and executable
- Return-oriented programming (ROP) was developed to use existing code for attacks
- Other defenses were developed (e.g. ASLR or CFI) to defend against this, but they have disadvantages
- Sophisticated attacks such as Blind ROP (BROP) exist that can defeat ASLR
- However, randomization still raises the bar for an attacker
- Are there other changes we can make to make attacks harder?



The Problem

- Can obfuscation of binaries or memory layout increase the security of a system?
- Compiler flags, stack layout, calling conventions, and/or system calls



Approach

- Changing compiler flags somewhat trivial
- Want to make more significant changes to programs
- Requirement: modifiable compiler
- GCC and LLVM large, complicated
- Tiny C Compiler (TCC) much smaller and simpler
- Make changes to TCC's code generation



Progress

- Cloned the Tiny C Compiler (TCC) repository, compiled and tested
- Studying the source code to understand it
- Beginning to make basic changes



Future Plans

- Attempt to change the stack layout
- Attempt to change the function calling convention
- Test changes with a web server application



Future Plans: Change Stack Layout

- Allocate local variables in different order
- Change local variable order at runtime
- Insert random padding in stack



Future Plans: Change Calling Convention

- Statically change argument order
- Dynamically shuffle argument order?



Future Plans: Test with Server

- Popular, moderately complex non-graphical program
- Nginx, Lighttpd



Challenges

- TCC has very little documentation
- Preserving correct behavior despite changes
- Creating enough variation to increase security
- Discovering bugs in software introduced due to assumptions on the underlying memory layout



Questions?

Erick Bauman - exb131030@utdallas.edu

Huibo Wang - hwx142830@utdallas.edu

Travis Wright - tnw130030@utdallas.edu